

ARC-AGI-3 Part 1:形式化与方法综述

Bo Hou · 2026-10-04 · 系列:ARC-AGI-3 世界模型推导

本篇是系列第一部分:先把问题形式化,统一符号,再用同一套符号重写现有全部方法,找出空白。后续部分将从第一性原理推导两层建模框架。

1 问题形式化

每个游戏是一台未知的确定性机器:agent 只看到渲染出的帧和少量元数据,规则与目标都不给,只能通过交互推断。观测字段已对照 arc-agi 0.9.9 / arcengine 源码核实。

1.1 符号表

符号	含义
$g \in G$	一个游戏; G_{pub} 为 25 个公开游戏, G_{priv} 为 110 个私有游戏
$\ell = 1, \dots, L_g$	游戏 g 的关卡序号, L_g 为关卡数(win_levels)
t	真实动作步数(全局计数)
$x_t \in \{0, \dots, 15\}^{64 \times 64}$	第 t 步后的帧(一步可能返回多帧, x_t 取最后一帧)
σ_t	游戏状态 $\in \{\text{NOT_FINISHED}, \text{WIN}, \text{GAME_OVER}\}$
c_t	已通过的关卡数(levels_completed)
$A_t \subseteq \{0, \dots, 7\}$	当前可用动作:0 = RESET,1-4 方向,5 交互,6 点击,7 撤销
a_t	动作; $a_t = 6$ 时带坐标 $(u, v) \in [0, 63]^2$
$o_t = (x_t, \sigma_t, c_t, L_g, A_t)$	完整观测
z_t	真实隐藏状态(含计时器、方向等画面外信息)
T^*, R^*	真实转移与渲染
$D_t = \{(x_i, a_i, x_{i+1})\}_{i < t}$	到第 t 步为止的交互记录
$\phi: x \mapsto s$	抽象映射,把帧变成物体级状态 s
$h \in H$	world model 假设: T_h (转移)、 G_h (目标判定)、可选 ϕ_h
Λ	外层先验(原语库、建模方法、预训练模型)
$b_{g,\ell}$	关卡 ℓ 的人类基线动作数(baseline_actions)
$n_{g,\ell}$	AI 在关卡 ℓ 上用掉的动作数

1.2 动力学假设

$$z_{t+1} = T^*(z_t, a_t), \quad x_t = R^*(z_t), \quad a_t \in A_t$$

- T^* 确定,但 z_t 可能不全显示在 x_t 中;Tycho 把它形式化为”参数化、渲染式的确定性 Moore 机”。
- 若同一 (x, a) 观察到多个 x' ,说明存在画面外状态, s 需要包含历史。
- 先验:只依赖 Core Knowledge(物体性、几何与拓扑、能动性;二手资料另列基础物理),不使用数字、字母等文化符号。

2 计分与全局目标

目标是用尽量少的真实动作通过尽量多、尽量靠后的关卡;动作超出人类基线时按平方衰减。单关得分(未通关为0):

$$r_{g,\ell} = \min(1.15, (\frac{b_{g,\ell}}{n_{g,\ell}})^2)$$

单局得分(按关卡序号加权)与总分:

$$S_g = \frac{\sum_{\ell=1}^{L_g} \ell r_{g,\ell}}{\sum_{\ell=1}^{L_g} \ell}, \quad S = \frac{1}{|G|} \sum_{g \in G} S_g$$

- $b_{g,\ell}$ 是首次接触玩家动作数的 upper median。
- $n_{g,\ell}$ 计入所有改变游戏状态的动作,探索动作同样计费;RESET 是否计入未核实。
- 1.15 的上限允许比人类少约 7% 的动作仍封顶($\sqrt{1.15} \approx 1.07$)。

全局目标:策略 π (含建模与规划)最大化期望总分,等价于在通关前提下最小化各关动作数:

$$\pi^* = \arg \max_{\pi} E_{g \sim G_{priv}} [S_g(\pi)]$$

3 两层建模框架

内层在单个游戏里现场建 world model 并规划;外层是离线学到、跨游戏通用的先验 Λ 。两层合起来是一个 meta-learning 问题。

3.1 内层:单游戏的建模-规划循环

每一步用 D_t 更新假设,再决定探索还是利用;动作永远在真实环境执行。

$$h_t = f_{\Lambda}(D_t), \quad a_t = \pi(h_t, x_t), \quad x_{t+1} = R^*(T^*(z_t, a_t))$$

1. 观察 x_t , 把 (x_{t-1}, a_{t-1}, x_t) 加入 D_t 。
2. 校验并修改 h : 要求对 D_t 中所有记录 T_h 与观察一致。
3. 规划: h 足够可信 $\rightarrow$ 在 T_h 内搜索通往 G_h 的动作序列; 否则 $\rightarrow$ 选一个最能区分候选假设的动作。
4. 执行时比对 T_h 的预测与真实 x_{t+1} ; 不一致回到第 2 步。

3.2 建模 cost

原始表述(方向层面): 最少的探测动作学到尽量准确的规则, 即 $\min C_{explore} + d(T^*, T_h)$ 。 T^* 不可观测, 可计算的版本用 D_t 上的数据拟合代替 d , 再加简洁性项, 得到 MDL 形式(δ 为 Kronecker delta, 两帧完全相同取 1):

$$L_{fit}(h; D_t) = \sum_{(x, a, x') \in D_t} (1 - \delta(T_h(x, a), x'))$$

$$h_t = \arg \min_{h \in H} L_{len}(h) + \lambda L_{fit}(h; D_t)$$

- L_{len} : 模型描述长度(代码越短越通用), 对应 Rodionov 的重构循环。
- 精确回放校验相当于硬约束 $L_{fit} = 0$ 。
- 求解不是梯度下降, 而是在程序空间 H 中的离散搜索: LLM 提出修改, 校验器筛选。

3.3 玩游戏 cost 与交替优化

规划目标是在 T_h 中以最少步数到达 G_h ;探索与规划交替,两者服从同一个全局目标(第 2 节):

$$\min n_{g,\ell} = n_{g,\ell}^{explore} + n_{g,\ell}^{play}$$

- 探索的价值应按” 它让 n^{play} 少了多少” 衡量:只学对过关有用的规则,而不是尽量多的规则。
- 何时值得花代价去建模、何时直接行动,Tycho 称为主动抽象(active abstraction)。

3.4 外层:跨游戏先验与 meta-learning

meta-learning 的一般形式:内层算法 f_ω 在任务的 D^{train} 上适应,在 D^{test} 上评估,外层优化 ω 。

$$\omega^* = \arg \min_{\omega} E_{T \sim p(T)} [L_T(f_\omega(D_T^{train}); D_T^{test})]$$

本题中 $\omega = \Lambda$,任务 = 游戏 g , D^{train} = 前面关卡的交互, D^{test} = 后面关卡:

$$\Lambda^* = \arg \max_{\Lambda} E_{g \sim G_{train}} [S_g(\pi_{f_\Lambda})]$$

层	学什么	何时学	比喻
外层 Λ	底层原语库、建模方法、预训练模型	离线(G_{pub} + 社区游戏);环境允许时评测中继续积累	身体素质、神经肌肉
内层 h	这个游戏的 T_h, G_h, ϕ_h	评测时,每个游戏现场	具体一套动作

- 私有集机制相对公开集是有意分布外的,所以 Λ 应存底层原语,而不是具体规则。
- 对比:Lux AI S3 的内层只需推断参数(规则已知);ARC 的内层要推断规则结构本身。

4 Kaggle 离线 / Preview 方法

离线条件下没有一家显式学 T_h :要么只学局部预测量,要么把 h 隐含在 LLM 上下文中; Λ 只有预训练先验或手写启发式。

4.1 StochasticGoose(Preview 第 1,12.58%)

内层只学” 画面会不会变” ,不学 T_h 与 G_h :

$$q_\theta(x, a) \approx P(\text{画面变化} | x, a), \quad \pi(a | x) \propto q_\theta(x, a)$$

- q_θ 为在线训练的 CNN,点击头输出 64×64 概率图;训练集是按 (x, a) 哈希去重、最多 20 万条的缓冲区;损失为二分类交叉熵。
- 局限:“变” $\neq$ “朝 G 前进”;有实验显示在现行计分下,即使 q_θ 完美,收益也几乎为零。

4.2 Blind Squirrel(Preview 第 2,6.71%)

h 是精确的查表模型(状态图),外加价值模型:

- 状态图:节点为哈希后的 x ,边为 (x, a, x') ;剪掉自环与成环动作。
- 得分提升后把本关状态按到里程碑的步数反标,训练 ResNet18 价值模型 $V_\psi(x, a)$,据此排序动作。
- 局限:查表不泛化,未见过的 x 无法预测。

4.3 Duck(Milestone 1 第 1;半私有 1.21%,公开 1.60 ± 0.45)

- Λ :Qwen 3.6 27B 的预训练先验。
- 内层:LLM 在 REPL 中把状态写成变量、自写代码分析; ϕ 为语义抽象,原始数字网格对 Python 工具隐藏;旧消息淘汰,只留 system prompt 与最近历史。 h 隐含在上下文和代码里,没有显式 L_{fit} 。
- 经验:harness 尽量轻,手工工具反而拖累。

4.4 Reki(Milestone 1 第 2,分数未查到)

- Λ :Gemma-4-31B + 手写点击先验(优先小的、颜色稀少的、像按钮的形状)。
- 内层: π_{LLM} 每步读最近几帧的标注渲染,输出变化描述、计划和接下来 1-4 个动作;反思记忆约每 10 步刷新;“点过从未引起变化”的物体类加入失效集合,本关不再点。

4.5 forge(Milestone 1 第 3,分数未查到)

与 Reki 同构,另加候选动作生成、打分仲裁与置信度提示;最佳运行恰好关掉了这些额外机制。

4.6 AERA(论文;公开 0.2116,私有 0.30)

Λ 为 Qwen2.5-0.5B,内层流程为探索 → 验证 → 规划;细节未读。

5 联网前沿方法(公开 25 游戏)

这一线都显式构建可执行的 T_h ,公开集已做到 RHAE 100;但全部是前沿 API 模型、公开集、无留出集评测, Λ 是”人写框架 + LLM 先验”,评测时不更新。

5.1 Rodionov EWM(GPT-5.5:58.12,完全解出 15 个;GPT-5.4:41.29,8 个)

- h :一份 Python 代码库,接口固定为 ϕ_h (状态表示)、 T_h (转移预测)、 G_h (目标判定)和规划器。
- 建模 cost:每次修改后跑校验器,要求 $L_{fit}(h; D_t) = 0$ (精确回放);随证据增加做重构以降低 $L_{len}(h)$,作者明说是 MDL 简洁性偏好的近似。
- 规划:在 T_h 中规划,执行时逐步比对预测,一旦不符立即停下修改 h 。
- 每次游玩从干净工作区开始,外层不累积。后续论文对比 ewma_sv(可执行 + 简化 + 回放校验)与 twma(纯文字 h)。

5.2 Tycho(Opus 5 / GPT-5.6 Sol:100.00;Opus 4.8:88.49,无模型基线 79.07)

- 形式化:参数化、渲染式的确定性 Moore 机;把可行动的帧与动画帧、过关帧、失败帧分开。
- h :每个游戏现场生成并修改的模拟器; Λ 是固定的 prompt、证据接口、actor-builder 协议、校验器和规划器。
- 关键变量:何时花代价建模、何时用模型。四种调度策略中”由 actor 主动请求 builder”最好;自动修复后的模型更准,RHAE 反而下降。
- 成本:Opus 5 一次约 \$2.99k。

5.3 OPINE-World(Opus 4.8:78.40,Tycho 文中同模型对比)

- ϕ :把 x_t 解析为物体集合 $s_t = \{o^i\}$,每个物体 $o^i = (\kappa^i, \tau^i, v^i)$ 。

- 学习:配对 s_t 与 s_{t+1} 中的物体,算效应签名,填入效应表行 (τ^i, a_t, u^i);用 Dirichlet 行诊断、类型不确定性和”本体误差”衡量抽象是否够用。
- 本体稳定后才合成可执行的 T_h ,并以完整下一状态相等来校验。

5.4 Kepler(Opus 5:100.00)

“可审计的 world model” ;Opus 5 一次约 \$778(按 2026-09-01 API 标价折算),token 量比 Tycho 少 36%。方法细节未读。

5.5 Symbolica Arcgentica(发布首日 36.08)

通过代码与游戏交互的递归 agent 编排; h 隐含,无显式建模 cost。

5.6 ARA(AgentNativeResearchLab,分数未查到)

边玩边构建结构化 h ,冷启动解不开时查询;保留带置信度的预测账本(每次查询 h 的预测与事后对错)。

6 其他方法与背景工作

ARC-AGI-3 上还有几种无正式成绩的做法;背景工作按”作用在内层还是外层”归类,出处多为记忆,标注”记忆”。

6.1 ARC-AGI-3 上的其他做法

方法	层	用本文符号描述	备注
Graph-Based Exploration(Rudakov, Shock, Cowley)	内层	h = 状态图,与 4.2 同路线	arXiv 2512.24156
游戏分类型开源 solver	外层	Λ = 手写游戏类别 + 每类专用解法; ϕ 用连通分量分割物体	地址未记录
本地 LLM 写 Python world model(设计稿)	内层	本地 LLM 生成 T_h ,用 D_t 校验,模型内规划,关卡间复用	只有设计,无成绩
自写模拟器 harness	内层	规则稳定后 agent 自写 sim.py 作为 T_h ,不花真实动作	无成绩记录

6.2 背景方法

方法	层	学什么	与本题的关系	出处
EMPA(Tsividis et al.)	内层	H = VGDL 规则空间上的贝叶斯推断 + 探索 + 规划	ARC 效率计分的灵感;VGDL 只能表达简单碰撞类交互	ARC 官方引用
TheoryCoder	内层	LLM 合成代码表达 T_h ,用于双层规划	突破 VGDL 表达力	TMLR 2025
WorldCoder(Tang, Key, Ellis)	内层	写代码 + 交互构建 T_h	程序式 world model 早期工作	2024
Code World Models + MCTS(Dainese et al.)	内层	LLM 生成代码 T_h ,MCTS 引导	同上	2024

方法	层	学什么	与本题的关系	出处
PoE-World	内层	T_h = 程序专家的乘积	组合式规则表示	2025
DreamCoder	外层	从多任务学可复用的程序原语库 Λ	“原语库作为先验”最接近的范式	记忆
Voyager	外层	agent 技能库持续积累	技能库跨任务复用	记忆
Dreamer / MuZero	内层	神经网络 T_h + 在想象中规划 / 搜索	需要大量交互,不适合首次接触的少样本设定	记忆
MAML / RL ²	外层	ω = 初始化参数 / 会适应的 RNN 权重	meta-learning 的标准形式(3.4 节)	记忆

7 对照总表

成绩越高的方法,内层越接近”显式 $T_h + L_{fit}$ 校验 + 模型内规划”;没有一家显式学外层 Λ 。

方法	条件	成绩	外层 Λ	内层 h	建模 cost	玩游戏 cost
Tycho	联网,Opus 5 / GPT-5.6 Sol	公开 100.00	固定框架 + LLM	可执行 T_h	L_{fit} 校验	模型内规划;建/用模型时机由 actor 决定
Kepler	联网,Opus 5	公开 100.00	可审计 WM 框架 + LLM	未读	未读	未读
Tycho	联网,Opus 4.8	公开 88.49	同上	同上	同上	同上
OPINE-World	联网,Opus 4.8	公开 78.40	物体解析 + 效应表	本体稳定后合成 T_h	Dirichlet 诊断 + 本体误差;下一状态相等	未读
Rodionov EWM	联网,GPT-5.5	公开 58.12	固定接口 + 校验器 + 重构	Python 的 ϕ_h, T_h, G_h	$L_{fit} = 0$ + 降低 L_{len} (MDL)	模型内规划,执行比对
Symbolica	联网,前沿模型	公开 36.08	LLM	隐含	无显式	无显式
StochasticGoose	Preview	12.58%	无	$q_\theta(x, a)$	二分类交叉熵	$\pi \propto q_\theta$
Blind Squirrel	Preview	6.71%	无	状态图 + V_ψ	无(精确记录)	价值回归
Duck	Kaggle 离线	半私有 1.21%	Qwen 3.6 27B	隐含于上下文 / 代码	无显式	无显式
AERA	论文	私有 0.30	Qwen2.5-0.5B	探索 → 验证 → 规划	未读	未读
Reki	Kaggle 离线	未查到	Gemma-4-31B + 点击先验	反思记忆 + 失效集合	无显式	1-4 步计划
forge	Kaggle 离线	未查到	同 Reki	同 Reki	无显式	无显式
ARA	联网,Opus 4.8 等	未查到	LLM	结构化 h	预测账本	未读

注:联网成绩是公开集上的 RHAE(百分制),离线与 Preview 成绩口径不同,不能直接横比。

8 空白与开放问题

两处空白:显式的外层 Λ (离线学原语库,评测中可更新),以及离线本地模型条件下的显式 T_h 。“怎么学”(探索选哪个动作、假设如何取舍)目前全靠 LLM 隐式决定。

- Milestone 1 第 2、3 名分数;Milestone 2(9/30 截止)结果
- Kepler、AERA、ARA 的方法细节与 cost 定义
- Kaggle 是否在同一进程顺序跑 110 个游戏;规则是否允许评测中跨游戏累积 Λ
- 规则是否允许用 API 模型生成的数据训练本地模型
- RESET 是否计入 $n_{g,\ell}$

Sources

- Executable World Models for ARC-AGI-3 (arXiv 2605.05138)
- Do Coding Agents Need Executable World Models... (arXiv 2607.15439)
- Tycho (arXiv 2607.28287) · 代码
- Kepler (arXiv 2610.00834)
- OPINE-World (arXiv 2607.01531)
- Rodionov 代码与运行产物
- Duck harness
- ARA 轨迹示例
- Awesome Code World Model